

2-6 死锁



1. 死锁定义

- 基本定义
 - 死锁：如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的。
 - 进程组特点：组内进程互相等。
 - 等待对象：等待的事件只能由同组其他进程引发。
 - 结果：所有相关进程都无法继续推进。
- 哲学家抢筷子
 - 持有资源：每个哲学家都拿着自己左边的筷子。
 - 等待资源：每个哲学家都在等待右边的筷子。
 - 不主动释放：所有哲学家都不会主动释放自己左边的筷子。
 - 结果：形成死锁，所有人都无法进餐。
- 饥饿
 - 含义：当等待时间给进程推进带来明显影响时，称发生饥饿。
 - 等待对象：可能是等 CPU，也可能是等 I/O 资源等。
 - 与死锁关系：饥饿不代表系统已经死锁。
 - 关键表现：至少有一个进程的执行被无限推迟。
- 死锁产生原因
 - 竞争不可抢占资源：系统中的不可抢占资源数量不足以满足多个进程运行需要。
 - 竞争结果：进程运行过程中因竞争资源陷入僵局。
 - 可抢占资源：对可抢占资源的竞争不会引起死锁。
 - 进程推进顺序不当：若资源申请或进程推进顺序安排不当，也可能产生死锁。

2. 死锁必要条件

- 互斥条件
 - 资源占用：一段时间内，进程分配到的资源只能被自己占用。
 - 其他进程请求：若其他进程此时请求该资源，只能等待。
- 请求和保持条件
 - 已持有资源：进程已经保持了至少一个资源。
 - 继续请求：进程又提出新的资源请求。
 - 请求阻塞：新请求的资源已被其他进程占有时，请求进程被阻塞。
 - 保持不放：被阻塞时，进程仍保持已有资源。
- 不可抢占条件
 - 资源不可抢占：进程已获得的资源在未使用完之前不能被抢占。
 - 释放方式：只能在进程使用完时由自己释放。
- 循环等待条件
 - 循环链：发生死锁时，必然存在一个进程-资源的循环链。
 - 等待形式： P_1 等 P_2 占有的资源， P_2 等 P_3 占有的资源，依次形成循环。

3. 处理方法

- 四类方法
 - 预防死锁：破坏死锁产生的必要条件。
 - 避免死锁：资源分配过程中防止系统进入不安全状态。
 - 检测死锁：允许死锁发生，再通过检测找出死锁。
 - 解除死锁：检测到死锁后采取措施恢复系统。
- 基本思想
 - 方法：设置某些限制条件。
 - 目标：破坏产生死锁四个必要条件中的一个或几个。
 - 实现特点：比较容易实现。
 - 互斥条件：通常不能破坏，因为临界资源本身就必须互斥使用。

4. 预防死锁

- 破坏请求和保持条件
 - 方法一：让所有进程在开始运行前，一次性申请整个运行过程中需要的全部资源。
 - 分配规则：若系统有足够资源，就一次性分配给进程。
 - 不能满足时：只要有一种资源不能满足，系统就不给该进程其他任何资源。
 - 破坏效果：运行期间不再提出新的资源请求，也不会保持一部分资源再等待另一部分资源。
 - 方法二：进程先申请并获得初期运行所需资源。
 - 再次申请条件：只有释放所有已分配资源以后，才允许申请新的资源。
 - 方法二特点：可提高设备利用率。
- 破坏不可抢占条件
 - 处理规则：已保持某些不可抢占资源的进程，若新的资源请求不能满足，必须释放已经保持的所有资源。
 - 重新申请：需要资源时再重新申请。
 - 代价：实现复杂，并且需要付出很大代价。
- 破坏循环等待条件
 - 资源排序：对系统资源进行线性排序。
 - 申请规则：每个进程必须按资源序号递增的顺序请求资源。
 - 反向请求处理：若拥有高序号资源后又想要低序号资源，需要先释放所有高序号资源，再申请低序号资源。
 - 破坏效果：破坏循环等待条件。
 - 限制：资源编号会限制新类型设备增加；作业使用顺序与系统预期不同也会导致资源浪费。

5. 避免死锁

- 基本思想
 - 性质：也是事先预防策略。
 - 方法：在资源分配过程中防止系统进入不安全状态。
 - 目标：避免发生死锁。
 - 特点：限制条件较少，系统性能较好。
- 安全状态
 - 安全状态：如果系统能按某种进程推进顺序为每个进程分配所需资源，满足每个进程最大需求，并让每个进程顺利完成，则系统处于安全状态。
 - 安全序列：合法分配资源的进程推进顺序，如 $(P_1, P_2, \dots, P_n)$ 。
 - 安全状态结论：系统处于安全状态时，不会进入死锁状态。
- 不安全状态
 - 不安全状态：如果找不到安全序列，系统处于不安全状态。
 - 风险：进入不安全状态后可能发生死锁。
 - 注意：不安全状态不一定已经发生死锁。
- 银行家算法
 - 核心目标：判断资源分配后系统是否仍处于安全状态。
 - 需求计算： $Need = 最大需求 - 已分配资源$ 。
 - 判断步骤：寻找 $Need \leq 可用资源$ 的进程。
 - 模拟完成：若某进程可完成，则释放它已分配的资源，加入可用资源。
 - 继续推进：重复寻找可完成进程。
 - 安全结论：若所有进程都能完成，则存在安全序列，系统处于安全状态。
 - 不安全结论：若无法让所有进程完成，则找不到安全序列，系统处于不安全状态。

6. 检测死锁

- 资源分配图
 - 图表示：由一组结点 N 和一组边 E 组成，记为 $G = (N, E)$ 。
 - 结点集合： N 分为两个互斥子集。
 - 进程结点：一组进程节点 P 。
 - 资源结点：一组资源节点 R 。
 - 边的连接：每条边都连接 P 和 R 中的一个结点。
 - 资源请求边：进程指向资源，表示进程想申请该资源。
 - 资源分配边：资源指向进程，表示资源已经分配给该进程。
- 资源分配图化简
 - 第一步：找出一个不阻塞也不独立的进程结点 P_i 。
 - 检查资源：判断能否给该进程所有需要的资源，使其运行完毕。
 - 可运行时：释放它占用的所有资源。
 - 图上操作：消去该进程的所有请求边和分配边。
 - 重复处理：把图中所有进程依次按这种方法化简。
 - 可完全化简：若能把所有边消除，图是可简化的。
 - 不可完全化简：若消除不了所有边，则系统处于死锁状态。

7. 解除死锁

- 资源剥夺法
 - 方法：挂起一些死锁进程。
 - 资源处理：抢占这些进程的资源，分配给其他死锁进程。
- 撤销进程法
 - 方法：强制撤销部分或全部死锁进程。
 - 资源处理：撤销进程并剥夺其资源。
 - 选择依据：可按优先级和撤销代价考虑撤销顺序。
 - 特点：实现简单，但代价较大。
- 进程回退法
 - 方法：让一个或多个死锁进程回退到足以回避死锁的地步。
 - 资源释放：进程回退时自愿释放资源，而不是被剥夺。
 - 系统要求：需要保存进程历史信息，并设置还原点。